

CHOFYAI STUDIO

05 · Referencia técnica

Documento	05-technical-reference.md
Versión analizada	0.5.1 (commit f840055)
Fecha del análisis	2026-08-28
Fuente Markdown	docs/system-documentation/05-technical-reference.md
Generado por	scripts/docs/build-pdf.mjs

Contenido

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

1. Comandos Tauri
 - 1.1 Sistema y rutas
 - 1.2 Herramientas
 - 1.3 Modelos
 - 1.4 Procesos huérfanos y diagnóstico
 - 1.5 Workflows y marketplace
2. Estructuras de datos
 - 2.1 DTOs de ``models.rs``
 - 2.2 Structs definidas en ``system.rs``
 - 2.3 Esquema de manifiesto (`RawManifest``)
 - 2.4 Tipos TypeScript adicionales
3. Funciones internas de Rust
 - 3.1 Rutas y configuración
 - 3.2 Manifiestos y plataforma
 - 3.3 Procesos
 - 3.4 Modelos y utilidades
 - 3.5 Lectura de estadísticas del sistema
4. Frontend
 - 4.1 ``src/utils.ts``
 - 4.2 ``src/i18n.ts``
 - 4.3 Helpers de ``src/App.tsx``
5. Constantes, claves y enumeraciones
6. Eventos
7. Variables de entorno
8. Puertos y URLs
9. Comandos de línea de órdenes
10. Índice de mensajes de error del backend

Estado: completo · Última revisión: 2026-08-27 · Versión analizada: 0.5.1 (commit f840055)

Catálogo de consulta rápida: firmas, tipos, constantes, eventos, rutas y mensajes de error. Para entender *por qué* el código hace lo que hace, véase [06 · Explicación profunda del código](#); para localizar archivos, [04 · Mapa del código](#).

Convención de nombres de parámetro: Tauri convierte los parámetros `snake_case` de Rust a `camelCase` al invocarlos desde JavaScript. En este documento se indican ambos.

1. Comandos Tauri

Los 35 comandos están registrados en el `invoke_handler` de `src-tauri/src/lib.rs` y todos se implementan en `src-tauri/src/system.rs`. Los parámetros `app: AppHandle` y `registry: State<'_, ProcessRegistry>` los inyecta Tauri: **no** se pasan desde JavaScript.

1.1 Sistema y rutas

Comando	Parámetros desde JS	Retorno	Efectos secundarios
<code>get_system_summary</code>	—	<code>SystemSummary</code>	Puede crear directorios y montar el sparsebundle vía <code>resolve_effective_home</code>
<code>save_studio_home</code>	<code>studioHome: string</code>	<code>AppSettings</code>	Escribe <code>settings.json</code>
<code>save_path_settings</code>	<code>modelsDir?</code> , <code>outputsDir?</code> , <code>cacheDir?</code>	<code>AppSettings</code>	Escribe <code>settings.json</code> ; cadena vacía limpia el override
<code>get_effective_paths</code>	—	<code>EffectivePaths</code>	Puede crear directorios
<code>list_volume_candidates</code>	—	<code>VolumeCandidate[]</code>	Escribe una sonda temporal para probar permisos
<code>get_system_stats</code>	—	<code>SystemStats</code>	Ejecuta <code>sysctl</code> , <code>vm_stat</code> , <code>top</code> , <code>df</code>

Detalles que importan:

- `save_studio_home` normaliza la entrada: si llega vacía o sólo con espacios, guarda `default_studio_home()`, es decir `$HOME/ChofyAIStudio`. **No** valida que la ruta exista ni que sea escribible.
- `save_path_settings` aplica `normalize(): Some("")` y `Some(" ")` se guardan como `None`.
- `list_volume_candidates` es el único comando que **no** devuelve `Result`: nunca falla, como mucho devuelve una lista corta.

1.2 Herramientas

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

Comando	Parámetros desde JS	Retorno	Efectos secundarios
list_tools	—	ToolSummary[]	Lee todos los YAML de apps/
install_tool	toolId: string	ActionResult	Lanza el script, emite eventos, escribe log
update_tool	toolId: string	ActionResult	Igual, pero exige que ya esté instalada
start_tool	toolId: string	ActionResult	Mata ocupantes del puerto, lanza proceso, registra PID
stop_tool	toolId: string	ActionResult	kill -TERM y quita el PID del registro
restart_tool	toolId: string	ActionResult	stop + espera 800 ms + start
health_check_tool	toolId: string	HealthResult	Puede limpiar un PID muerto del registro
open_tool_directory	toolId: string	ActionResult	Crea el directorio si falta y lo abre con open
open_tool_log	toolId: string	ActionResult	Abre run.log o, si no existe, install.log
read_tool_log	toolId, kind, lastLines?	string	Sólo lectura
relocate_module	toolId, targetDir	ActionResult	Mueve o copia el directorio y escribe settings.json
clear_module_override	toolId: string	AppSettings	Escribe settings.json ; no mueve archivos
list_running_pids	—	Record<string, number>	Sólo lectura del registro en memoria

Ficha · install_tool

```
#[tauri::command]
pub fn install_tool(app: AppHandle, tool_id: String) -> Result<ActionResult, String>
```

- find_manifest → error No se encontro manifest para <id> si no existe.
- load_settings + resolve_effective_home .
- Delega en run_install_script , que valida plataforma, resuelve el script, lo ejecuta con bash (o pwsh en Windows), emite install-progress por línea, escribe <studio_home>/logs/<tool>-install.log , emite install-done y revalida installed_if .

Errores posibles, con el texto literal:

Mensaje	Causa
ChofyAI Studio · 05 · Referencia técnica <id> no soporta la plataforma actual (<key>). Plataformas soportadas: [...]	v0.5.1 (commit f840055) · 2026-08-28 platforms: no incluye la plataforma actual
<id> no declara install_script para <key>	Ni install_scripts[key] ni install_script
No existe script: <ruta>	El script referenciado no está en disco (caso real de Linux)
Instalación de <name> terminó pero faltan artefactos: <lista>. Revisa <log>	El script salió con 0 pero installed_if no se cumple
La instalacion fallo para <name>. Revisa <log>	Código de salida distinto de 0

Riesgo al modificar: es el comando con más efectos colaterales del sistema. Cualquier cambio en el orden de emisión de eventos rompe la cola de instalación del frontend, que depende de que `install-done` llegue después del último `install-progress`.

Ficha · `start_tool`

```
#[tauri::command]
pub fn start_tool(app: AppHandle, tool_id: String,
    registry: State<'_, ProcessRegistry>) -> Result<ActionResult, String>
```

Orden de operaciones: resolver manifiesto y rutas → comprobar que el directorio existe → validar `installed_if` → **pre-flight de puerto** (`lsof -ti :PORT -sTCP:LISTEN`, y `kill -9` a todo PID que no esté en el registro) → crear el log → `spawn` de `bash -lc "<run.command>"` con `current_dir` en el directorio de instalación y `stdout/stderr` redirigidos al log → registrar el PID y persistirlo.

Mensaje de error	Causa
<id> no tiene run.command para <key>	El manifiesto no define comando para la plataforma
No existe la ruta de instalacion: <ruta>	El directorio no está
<name> no está instalado correctamente. Faltan: <lista>. Reinstala desde la UI.	Falla <code>installed_if</code>

`ActionResult.opened_url` se rellena con `http://127.0.0.1:<default_port>` sólo si el manifiesto declara puerto.

Riesgo al modificar: el `kill -9` del pre-flight afecta a procesos **ajenos** a la aplicación. Véase [11 · Seguridad](#).

Ficha · `relocate_module`

```
#[tauri::command]
pub fn relocate_module(app: AppHandle, tool_id: String,
    target_dir: String) -> Result<ActionResult, String>
```

Validaciones, en orden: la ruta debe ser absoluta; el destino no puede ser igual al origen; si el destino existe y no está vacío, aborta; si existe vacío, lo borra para que `rename` funcione; el padre debe ser escribible. Después intenta `fs::rename` y, si falla (típico entre volúmenes distintos), cae a `copy_dir_recursive` + `remove_dir_all`. Finalmente inserta el override en `settings.tool_overrides` y guarda.

Mensaje	Causa
La ruta de destino debe ser absoluta.	Ruta relativa
El destino es igual al origen.	Sin cambio
El destino ya existe y no está vacío: <ruta>	Riesgo de mezclar instalaciones
Sin permisos de escritura en <ruta>	Padre no escribible
Copia falló: <error>	Fallo durante la copia entre dispositivos

Riesgo: la copia no es transaccional. Si falla a mitad, quedan archivos en ambos lados.

Ficha · `read_tool_log`

```
#[tauri::command]
pub fn read_tool_log(app: AppHandle, tool_id: String, kind: String,
                    last_lines: Option<usize>) -> Result<String, String>
```

`kind` sólo acepta `"install"` o `"run"`; cualquier otro valor produce `kind inválido: <valor>`. `last_lines` por defecto es 500. Si el archivo no existe devuelve `(sin log <kind> aún en <ruta>)` — es un `Ok`, no un error.

1.3 Modelos

Comando	Parámetros desde JS	Retorno	Notas
<code>list_tool_models</code>	<code>toolId: string</code>	<code>ModelEntry[]</code>	Recorre <code><install_dir>/models</code> con profundidad máxima 3, ignora <code>._*</code> y <code>.DS_Store</code> , ordena por tamaño descendente
<code>list_declared_models</code>	<code>toolId: string</code>	<code>DeclaredModel[]</code>	Cruza <code>manifest.models</code> con lo que hay en disco; crea el directorio si falta
<code>download_tool_model</code>	<code>toolId</code> , <code>repoId</code>	<code>ActionResult</code>	Sólo permite repositorios declarados en el manifiesto
<code>delete_tool_model</code>	<code>toolId</code> , <code>relativePath</code>	<code>ActionResult</code>	Con guardia de path traversal

`delete_tool_model` rechaza con `relative_path inválido` si la ruta está vacía o contiene `..`; con `path traversal bloqueado` si tras `canonicalize` el destino queda fuera del directorio de modelos; y con `solo se borran archivos` si el destino es un directorio.

`download_tool_model` rechaza con `'<repo>'` no está declarado en el manifest de `<id>` y requiere que exista `scripts/mac/download-hf-model.sh`, o devuelve `No existe el helper: <ruta>`

1.4 Procesos huérfanos y diagnóstico

Comando	Parámetros desde JS	Retorno	Notas
<code>list_orphan_ports</code>	—	<code>OrphanPort[]</code>	Recorre los puertos declarados por los manifiestos con <code>lsof -nP -iTCP:<port> -sTCP:LISTEN -Fpc</code>
<code>adopt_orphan</code>	<code>toolId</code> , <code>pid</code>	<code>ActionResult</code>	Falla con <code>PID <n></code> no está vivo si el proceso ya murió
<code>kill_orphan</code>	<code>pid: number</code>	<code>ActionResult</code>	Envía <code>SIGTERM</code> ; siempre devuelve <code>ok: true</code>
<code>run_doctor</code>	<code>studioHome?: string</code>	<code>string</code>	Ejecuta <code>scripts/mac/doctor.sh</code> ; devuelve stdout y, si lo hay, un bloque <code>--- stderr ---</code>
<code>append_crash_log</code>	<code>message: string</code>	<code>string</code> (ruta)	Añade una línea con marca de tiempo Unix
<code>read_crash_log</code>	—	<code>string</code>	Devuelve las últimas 200 líneas; cadena vacía si no existe
<code>notify_macos</code>	<code>title</code> , <code>body</code>	<code>void</code>	<code>osascript -e 'display notification ...'</code> ; escapa comillas y saltos de línea

`run_doctor` falla con `doctor.sh` no existe en `<ruta>` si el script no está.

1.5 Workflows y marketplace

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

Comando	Parámetros desde JS	Retorno	Notas
list_workflows	—	unknown[] (JSON)	Lee workflows/*.yaml y *.yml, ignora ._, ordena por id
save_workflow	id, yamlContent	ActionResult	Valida el id y exige los campos id, name, description, steps
delete_workflow	id: string	ActionResult	Falla con workflows/<id>.yaml no existe
list_marketplace_tools	—	MarketplaceEntry[]	Lee marketplace/registry.yaml; devuelve lista vacía si no lo encuentra
import_marketplace_tool	id: string	ActionResult	Genera apps/<id>.yaml mínimo; nunca sobrescribe

Validación de id en validate_workflow_id:

Mensaje	Regla
id vacío	Longitud cero
id no puede contener / \ ni ..	Separadores o recorrido de rutas
id solo permite [a-zA-Z0-9_-]	Cualquier otro carácter

save_workflow rechaza además con YAML inválido: <error>, YAML root debe ser un mapping y falta campo obligatorio: <campo>.

import_marketplace_tool falla con Tool '<id>' no está en el marketplace y con Ya existe apps/<id>.yaml – no se sobrescribe.

2. Estructuras de datos

2.1 DTOs de `models.rs`

Struct	Campos	Serializa hacia
SystemSummary	app_name, app_version, os, arch, studio_home, studio_home_effective, using_fallback, settings_file, platform_key, platform_support	SystemSummary en <code>types.ts</code>
ToolSummary	file_name, id, name, icon, category, runtime, description, recommended, default_port, install_dir, install_script, run_command, installed, installed_checks, missing_checks, relocated	ToolManifest
AppSettings	studio_home, tool_overrides, fallback_home, sparsebundle_path, models_dir, outputs_dir, cache_dir	AppSettings
HealthResult	tool_id, running, port_open, pid	HealthResult
InstallEvent	tool_id, line	InstallEvent
VolumeCandidate	path, label, kind, mounted, writable, free_bytes, total_bytes	VolumeCandidate
SystemStats	cpu_usage, cpu_cores, mem_used_bytes, mem_total_bytes, disk_free_bytes, disk_total_bytes, disk_path, uptime_secs, load_avg_1m	SystemStats

Campos con `#[serde(default)]` en `AppSettings`: `tool_overrides`, `fallback_home`, `sparsebundle_path`, `models_dir`, `outputs_dir`, `cache_dir`. Es lo que permite leer un `settings.json` antiguo sin romper. `studio_home` **no** lo tiene: si falta, la deserialización falla y `load_settings` devuelve los valores por defecto completos.

En `SystemSummary`, `platform_key` y `platform_support` también llevan `#[serde(default)]`.

2.2 Structs definidas en system.rs

Struct	Campos	Uso
ProcessRegistry	Mutex<HashMap<String, u32>>	Estado gestionado por Tauri: tool_id → PID
ActionResult	ok, message, log_path, opened_url	Retorno común de las acciones
EffectivePaths	studio_home, models_dir, outputs_dir, cache_dir	Rutas ya resueltas para la interfaz
ModelEntry	name, relative_path, absolute_path, size_bytes, modified_secs	Archivo de modelo en disco
DeclaredModel	repo_id, local_name, local_path, present, size_bytes	Modelo declarado en el manifiesto
OrphanPort	tool_id, tool_name, port, pid, command	Proceso no registrado que ocupa un puerto conocido
MarketplaceEntry	id, name, category, runtime, short_description, homepage, repo, default_port, estimated_size_gb, requires, install_hint, notes	Entrada del catálogo
MarketplaceFile	tools: Vec<MarketplaceEntry>	Raíz de registry.yaml
AppPaths	apps_dir, settings_path	Resultado de app_paths()
RawManifest	ver tabla siguiente	Deserialización de apps/*.yaml
RawRun	command, commands	Bloque run: del manifiesto

2.3 Esquema de manifiesto (RawManifest)

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

Campo YAML	Tipo Rust	Obligatorio	Comportamiento
id	String	sí	Clave única; se usa en rutas y en el registro
name	String	sí	Nombre visible
icon	Option<String>	no	Emoji mostrado en la tarjeta
category	String	sí	El validador de CI acepta voice , asr , video , image , music , system
runtime	String	sí	CI acepta python , binary , node , mlx , mixed
description	Option<String>	no en Rust, sí en CI	Vacío si falta
recommended	Option<bool>	no	false por defecto
default_port	Option<u16>	no	Habilita health check por puerto, Ver UI y detección de huérfanos
studio_home_subdir	Option<String>	no	Por defecto tools/<id>
install_script	Option<String>	no	Fallback mono-plataforma
install_scripts	Option<HashMap<String ,String>>	no	Tiene precedencia sobre el anterior
installed_if	Option<Vec<String>>	no en Rust, sí en CI	Rutas relativas al directorio de instalación
run	Option<RawRun>	no	Sin él, la herramienta no puede arrancarse
platforms	Option<Vec<String>>	no	Si falta se asume mac-arm64
models	Option<Vec<String>>	no	Repositorios de Hugging Face descargables desde la interfaz

Campos presentes en los YAML del repositorio que **RawManifest no declara** y que, por tanto, `serde_yaml` ignora silenciosamente: `python_manager` , `healthcheck` , `install` , `notes` . Están registrados como deuda en [15 · Riesgos y deuda técnica](#).

2.4 Tipos TypeScript adicionales

Definidos en `src/types.ts` y sin contraparte directa en Rust:

Tipo	Para qué
ChofyAI Studio · 05 · Referencia técnica QueueItem , QueueStatus	v0.5.1 (commit f840055) · 2026-08-28 Estado de la cola de instalación en el frontend
WorkflowDef , WorkflowInput , WorkflowStep	Forma esperada de los YAML de workflows/
Toast , ToastKind	Notificaciones internas
ModelDownloadProgress , ModelDownloadDone	Payload de los eventos de descarga
EffectivePaths , MarketplaceEntry , ModelEntry , DeclaredModel	Espejos de structs de system.rs

Diferencias detectadas entre Rust y TypeScript:

- ToolManifest (TS) marca relocated como opcional; ToolSummary (Rust) siempre lo envía.
- category y runtime son uniones cerradas en TypeScript pero String libres en Rust: un manifiesto con una categoría inventada compila en Rust y sólo lo detiene el validador de CI.
- WorkflowDef no tiene equivalente en Rust: list_workflows devuelve serde_json::Value sin validar contra ese esquema.

3. Funciones internas de Rust

3.1 Rutas y configuración

Función	Firma resumida	Qué hace	Riesgo al modificar
repo_root()	-> Option<PathBuf>	Detecta si se está ejecutando dentro del repositorio (exige apps/ , scripts/ y src-tauri/)	Cambia toda la resolución de rutas entre desarrollo y producción
app_paths(app)	-> Result<AppPaths, String>	Devuelve apps_dir y settings_path según el modo	Alto
settings_path(app)	-> Result<PathBuf, String>	Atajo sobre el anterior	Medio
script_path(app, rel)	-> Result<PathBuf, String>	Ruta del script en repo o en recursos del bundle	Alto
resolve_resource_path(app, rel)	-> Result<PathBuf, String>	BaseDirectory::Resource	Medio
home_dir()	-> PathBuf	\$HOME o %USERPROFILE%, con . como último recurso	Bajo
default_studio_home()	-> String	<home>/ChofyAISTudio	Medio
fallback_home_for(settings)	-> String	settings.fallback_home si no está vacío, si no el default	Bajo
path_is_usable(path)	-> bool	Directorio escribible, o padre montado y escribible	Alto
is_writable_dir(path)	-> bool	Escribe y borra .chofyai-write-probe	Medio: escribe en disco del usuario
resolve_effective_home(settings)	-> String	Ruta solicitada → auto-montaje del sparsebundle → fallback	Muy alto
load_settings(app)	-> Result<AppSettings, String>	Lee y deserializa; ante cualquier fallo devuelve valores por defecto	Alto: enmascara un JSON corrupto
save_settings_to_disk(app, s)	-> Result<(), String>	serde_json::to_string_pretty + fs::write	Escritura no atómica
ensure_parent(path)	-> Result<(), String>	create_dir_all del padre	Bajo
effective_models_dir /	(&AppSettings, &str) -> PathBuf	Override o subdirectorio por defecto	Medio

ChofYAI Studio - 05 - Referencia técnica

Función	Firma resumida	Qué hace	Riesgo al modificar
<code>effective_outputs_dir / effective_cache_dir</code>			v0.5.1 (commit f840055) · 2026-08-28
<code>apply_path_env(cmd, settings, home)</code>	<code>-> ()</code>	inyecta <code>CHOFYAI_MODELS_DIR</code> , <code>CHOFYAI_OUTPUTS_DIR</code> , <code>CHOFYAI_CACHE_DIR</code>	Medio
<code>log_dir(home)</code>	<code>-> PathBuf</code>	<code><home>/logs</code>	Bajo

3.2 Manifiestos y plataforma

Función	Firma resumida	Qué hace
<code>collect_manifests(app)</code>	<code>-> Result<Vec<(String, RawManifest)>, String></code>	Lee <code>apps/*.yaml</code> (profundidad 1) y ordena por nombre; un YAML inválido aborta toda la lista
<code>find_manifest(app, id)</code>	<code>-> Result<(String, RawManifest), String></code>	Busca por <code>id</code>
<code>manifest_install_dir(m, home, overrides)</code>	<code>-> PathBuf</code>	Override absoluto, override relativo bajo <code>home</code> , o <code>home/<studio_home_subdir></code>
<code>current_platform_key()</code>	<code>-> &'static str</code>	<code>win-x64</code> , <code>mac-arm64</code> , <code>mac-x64</code> , <code>linux-x64</code> o <code>unknown</code>
<code>resolve_install_script(m)</code>	<code>-> Option<String></code>	Prefiere <code>install_scripts[key]</code>
<code>resolve_run_command(run)</code>	<code>-> Option<String></code>	Prefiere <code>commands[key]</code>
<code>platform_supported(m)</code>	<code>-> bool</code>	Sin <code>platforms</code> : sólo es válido <code>mac-arm64</code>
<code>script_shell()</code>	<code>-> &'static str</code>	<code>pwsh</code> en Windows, <code>bash</code> en el resto
<code>shell_inline_command(cmd, s)</code>	<code>-> ()</code>	<code>-lc</code> en Unix, <code>-NoProfile -Command</code> en Windows

3.3 Procesos

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

Función	Firma resumida	Qué hace
<code>pid_is_alive(pid)</code>	<code>-> bool</code>	<code>kill -0 <pid></code>
<code>processes_state_path(app)</code>	<code>-> Result<PathBuf, String></code>	<code>processes.json</code> junto a <code>settings.json</code>
<code>persist_registry(app, map)</code>	<code>-> ()</code>	Escribe el mapa; errores silenciados
<code>restore_registry(app, registry)</code>	<code>-> ()</code>	Al arrancar, conserva sólo los PID vivos y reescribe el archivo
<code>run_install_script(app, id, m, home)</code>	<code>-> Result<ActionResult, String></code>	Núcleo de la instalación
<code>copy_dir_recursive(src, dst)</code>	<code>-> std::io::Result<()></code>	Copia con soporte de symlinks en Unix
<code>open_in_system(path)</code>	<code>-> Result<(), String></code>	<code>open</code> en macOS; en otras plataformas devuelve Esta accion solo esta disponible en macOS.

3.4 Modelos y utilidades

Función	Firma resumida	Qué hace
<code>resolve_models_dir(app, id)</code>	<code>-> Result<PathBuf, String></code>	<code><install_dir>/models</code> de esa herramienta
<code>safe_model_name(repo_id)</code>	<code>-> String</code>	Basename tras el último <code>/</code>
<code>dir_size(path)</code>	<code>-> u64</code>	Suma recursiva, sin límite de profundidad
<code>validate_workflow_id(id)</code>	<code>-> Result<(), String></code>	Ver tabla de la sección 1.5
<code>workflows_dir(app)</code>	<code>-> Result<PathBuf, String></code>	<code>workflows/</code> en repo o recursos

3.5 Lectura de estadísticas del sistema

Todas dependen de utilidades de macOS y devuelven cero cuando no pueden leer.

Función	Fuente	Detalle
ChofyAI Studio · 05 · Referencia técnica <code>run_capture(cmd, args)</code>	—	v0.5.1 (commit f840055) · 2026-08-28 Ejecuta y captura stdout como <code>String</code>
<code>read_cpu_cores()</code>	<code>sysctl -n hw.ncpu</code>	Número de núcleos
<code>read_mem_total()</code>	<code>sysctl -n hw.memsize</code>	Bytes totales
<code>read_mem_used()</code>	<code>vm_stat</code>	<code>total - (free + inactive + speculative) × page_size;</code> tamaño de página por defecto 16384
<code>parse_pages(s)</code>	—	Limpia puntos y comas del formato de <code>vm_stat</code>
<code>read_cpu_usage()</code>	<code>top -l 1 -n 0</code>	<code>100 - idle</code> , acotado a <code>0..100</code>
<code>read_load_avg()</code>	<code>sysctl -n vm.loadavg</code>	Primer valor de la terna
<code>read_uptime()</code>	<code>sysctl -n kern.boottime</code>	Ahora – arranque
<code>read_disk_usage(path)</code>	<code>df -k <path></code>	Devuelve <code>(total, disponible)</code> en bytes
<code>list_external_volumes()</code>	<code>/Volumes</code>	Todos los directorios montados allí

4. Frontend

4.1 `src/utils.ts`

```
export function fmtBytes(b?: number | null): string
export function fmtElapsed(ms: number): string
export function parseInstallLine(prev: LineParse, line: string): LineParse
```

- `fmtBytes` devuelve — para `0`, `null` y `undefined`; usa un decimal por debajo de 10 y ninguno por encima.
- `fmtElapsed` devuelve `M:SS` (los minutos no se rellenan con cero).
- `parseInstallLine` limpia códigos ANSI y aplica, en orden, estos patrones:

Patrón reconocido	Fase resultante	Porcentaje
ChofyAI Studio · 05 · Referencia técnica Clonando / Cloning into	Clonando repositorio	— v0.5.1 (commit f840055) · 2026-08-28
Receiving objects: N%	Descargando objetos git	N
Resolving deltas: N%	Resolviendo deltas	N
Creating virtual environment / Creando venv	Creando entorno Python	—
Downloading ... model, Downloading ggml, saved in *.bin	Descargando modelo	—
Resolved N packages, Installing collected, Downloading, Installed N packages	Instalando dependencias Python	—
[N%] al inicio de línea	Compilando (cmake/make)	min(N,100)
Linking CXX / Linking C seguido de espacio	Enlazando binarios	—
Formato de progreso de curl	—	N y velocidad
INSTALL_OK	Listo	100

Si ninguna coincide, se conservan los valores anteriores: por eso la firma recibe prev .

4.2 src/i18n.ts

```
export type Lang = 'es' | 'en'
export const SUPPORTED_LANGS: Lang[]
export const DEFAULT_LANG: Lang          // 'es'
export function getLang(): Lang
export function setLang(l: Lang): void
export function t(key: string, params?: Record<string, string | number>): string
export function useT(): (key: string, params?: ...) => string
export function knownKeys(): string[]
```

t() cae al diccionario por defecto si falta la clave y, si tampoco está, devuelve la clave cruda — comportamiento del que depende la prueba de paridad de i18n.test.ts —. Los parámetros se sustituyen con la sintaxis {nombre} . setLang ignora idiomas no soportados, persiste en localStorage bajo chofyai_lang y actualiza document.documentElement.lang .

4.3 Helpers de src/App.tsx

ChofyAI Studio · 05 · Referencia técnica

v0.5.1 (commit f840055) · 2026-08-28

Función	Firma	Notas
tauriInvoke<T>	`(cmd, args?, opts?) => Promise<T \	null>`
notify	(kind, title, body?) => void	Toast interno
notifyNative	(title, body) => Promise<void>	Comando notify_macos ; no hace nada fuera de Tauri
applyTheme	(theme: Theme) => void	Escribe document.documentElement.dataset.theme
substituteVars	(template, inputs) => string	Reemplaza {{ inputs.clave }}
runWorkflowStep	(step, inputs, files) => Promise<{ok, output?, error?}>	Ejecuta un paso HTTP o devuelve un stub
buildYaml	(meta, inputs, steps) => string	Serializa el workflow del constructor visual
emptyStep	() => BuilderStep	Paso en blanco
setToasterRef	(fn) => void	Conecta el Toaster montado con la función global notify

5. Constantes, claves y enumeraciones

Constante	Valor	Archivo
APP_VERSION	'0.5.0'	src/App.tsx
ONBOARDING_KEY	'chofyai_onboarding_done'	src/App.tsx
THEME_KEY	'chofyai_theme'	src/App.tsx
STORAGE_KEY (idioma)	'chofyai_lang'	src/i18n.ts
inTauri	'__TAURI_INTERNALS__' in window	src/App.tsx
APP_STARTED_AT	Date.now() al cargar el módulo	src/App.tsx
SHORTCUTS	11 entradas de atajos	src/App.tsx
CATEGORY_LABEL / CATEGORY_EMOJI	Etiqueta y emoji por categoría	src/App.tsx
fallbackTools	5 herramientas simuladas para el modo web	src/App.tsx

Enumeraciones efectivas:

Concepto	Valores
ChofyAI Studio · 05 · Referencia técnica category	v0.5.1 (commit f840055) · 2026-08-28 voice, asr, video, image, music, system
runtime	python, binary, node, mlx, mixed
platform_key	mac-arm64, mac-x64, win-x64, linux-x64, unknown
platform_support	validated (mac-arm64), experimental (win-x64), todo (linux-x64), unsupported
QueueStatus	pending, installing, done, failed
Estado de paso de workflow	pending, running, ok, fail, skipped
ToastKind	info, success, warn, error
Theme	dark, light, system
kind de VolumeCandidate	home, external, custom

6. Eventos

Evento	Payload	Emisor	Receptor
install-progress	InstallEvent { tool_id, line }	Hilo lector de stdout en run_install_script	Cola de instalación
install-done	InstallEvent { tool_id, line } – la línea empieza por OK: o ERROR:	Final de run_install_script	Cola, toasts y notificación nativa
model-download-progress	{ tool_id, repo_id, line }	download_tool_model	ModelsPanel
model-download-done	{ tool_id, repo_id, ok }	download_tool_model	ModelsPanel

El frontend distingue éxito de fallo en `install-done` comprobando el prefijo `OK:` . Es un contrato frágil basado en texto: cambiarlo en Rust rompe la interfaz sin error de compilación.

7. Variables de entorno

Variable	Quién la define	Quién la lee
CHOFYAI_STUDIO_HOME	run_install_script , start_tool , restart_tool , download_tool_model	resolve_studio_home en scripts/mac/common.sh
CHOFYAI_MODELS_DIR	apply_path_env	resolve_models_dir (Bash)
CHOFYAI_OUTPUTS_DIR	apply_path_env	resolve_outputs_dir (Bash)
CHOFYAI_CACHE_DIR	apply_path_env	resolve_cache_dir (Bash)
STUDIO_HOME	El usuario, manualmente	resolve_studio_home como alternativa
CHOFYAI_DISABLE_UV	El usuario	detect_uv : con valor 1 fuerza pip clásico
HF_HOME	scripts/mac/install-qwen3- tts.sh	Cliente de Hugging Face
UV_LINK_MODE	El mismo script (copy)	uv
CHOFYAI_CHROME	El usuario	findChrome() en scripts/docs/build-pdf.mjs
CHOFYAI_SKIP_MERMAID	El usuario	ensureMermaid() en el mismo script
GRADIO_SERVER_PORT , GRADIO_SERVER_NAME	El run.command de FaceFusion	Gradio

8. Puertos y URLs

Recurso	Valor	Origen
Qwen3-TTS	http://127.0.0.1:7860	apps/qwen3-tts.yaml
whisper.cpp	http://127.0.0.1:8178	apps/whispercpp.yaml
FaceFusion	http://127.0.0.1:7862	apps/facefusion.yaml
AceForge	http://127.0.0.1:7857	apps/aceforge.yaml
ComfyUI	http://127.0.0.1:8188	apps/comfyui.yaml
Servidor de desarrollo Vite	http://localhost:1420	vite.config.ts y devUrl de tauri.conf.json
Endpoint de transcripción	POST /inference (multipart)	workflows/transcribe-audio.yaml
Endpoint de encolado de ComfyUI	POST /prompt (JSON)	workflows/comfyui-prompt.yaml
Últimos releases	API de GitHub, consultada por UpdateChecker	src/App.tsx

9. Comandos de línea de órdenes

Comando	Qué hace
<code>pnpm dev:web</code>	Vite en modo desarrollo, sin backend
<code>pnpm build:web</code>	Compila el frontend a <code>dist/</code>
<code>pnpm preview</code>	Sirve el build del frontend
<code>pnpm tauri:dev</code>	Aplicación de escritorio completa
<code>pnpm tauri:build</code>	Empaqueta con la configuración por defecto
<code>pnpm tauri:build:app / :dmg / :mac</code>	Variantes de empaquetado
<code>pnpm package:mac</code>	<code>scripts/mac/build-release.sh</code>
<code>pnpm preflight:mac</code>	Verifica prerequisites de build
<code>pnpm test / pnpm test:watch</code>	Vitest
<code>pnpm test:rust</code>	<code>cargo test</code> en <code>src-tauri</code>
<code>node scripts/docs/build-pdf.mjs [prefijo]</code>	Genera los PDF de esta documentación
<code>bash scripts/mac/doctor.sh <ruta></code>	Diagnóstico del entorno
<code>bash scripts/mac/clean-appledouble.sh</code>	Borra archivos <code>._*</code>
<code>bash scripts/mac/cleanup-tool.sh <home> <id></code>	Elimina una herramienta instalada
<code>bash scripts/mac/mount-apfs.sh [punto]</code>	Monta un sparsebundle
<code>bash scripts/mac/download-hf-model.sh <repo> <destino></code>	Descarga un repositorio de modelos

10. Índice de mensajes de error del backend

Mensaje	Función de origen
No se encontro manifest para <id>	find_manifest
<id> no soporta la plataforma actual (<key>). Plataformas soportadas: [...]	run_install_script
<id> no declara install_script para <key>	run_install_script
No existe script: <ruta>	run_install_script
Instalación de <name> terminó pero faltan artefactos: <lista>. Revisa <log>	run_install_script
La instalacion fallo para <name>. Revisa <log>	run_install_script
<name> no está instalado. Usa Instalar primero.	update_tool
<id> no tiene run.command para <key>	start_tool , restart_tool
No existe la ruta de instalacion: <ruta>	start_tool
<name> no está instalado correctamente. Faltan: <lista>. Reinstala desde la UI.	start_tool
<id> no tiene proceso activo registrado	stop_tool (devuelve ok: false , no error)
relative_path inválido	delete_tool_model
path traversal bloqueado	delete_tool_model
solo se borran archivos	delete_tool_model
'<repo>' no está declarado en el manifest de <id>	download_tool_model
No existe el helper: <ruta>	download_tool_model
PID <n> no está vivo	adopt_orphan
kind inválido: <valor>	read_tool_log
No hay logs disponibles para <id>	open_tool_log
La ruta de destino debe ser absoluta.	relocate_module
El destino es igual al origen.	relocate_module
El destino ya existe y no está vacío: <ruta>	relocate_module
Sin permisos de escritura en <ruta>	relocate_module
Copia falló: <error>	relocate_module

Mensaje	Función de origen
ChofyAI Studio · 05 · Referencia técnica id vacío · id no puede contener / \ ni .. · id solo permite [a-zA-Z0-9_-]	validate_workflow_id v0.5.1 (commit f840055) · 2026-08-28
YAML inválido: <error> · YAML root debe ser un mapping · falta campo obligatorio: <campo>	save_workflow
workflows/<id>.yaml no existe	delete_workflow
Tool '<id>' no está en el marketplace	import_marketplace_tool
Ya existe apps/<id>.yaml – no se sobrescribe	import_marketplace_tool
doctor.sh no existe en <ruta>	run_doctor
Esta accion solo esta disponible en macOS.	open_in_system fuera de macOS

Todos los mensajes son cadenas libres: no hay códigos de error estructurados. Cambiar el texto de uno no rompe la compilación, pero sí puede romper cualquier comprobación que el frontend haga sobre él.